N052 로드밸런서로 무중단 배포 구성하기_학습서

Chapter01

로드밸런서로 무중단 배포 구성하기

이 교안은 에이전트 서비스를 저장소 루트의 Dockerfile로 이미지로 만들 수 있는 상태에서 시작합니다. 이 교안에서도 지금까지 배운 요소로 만든 AI 친구 에이전트 서비스를 예로 들어 실습하고, 줄여서 에이전트 서비스라고 부르겠습니다.

배포를 자동으로 돌리면 하루에도 여러 번 새 버전이 나갑니다. 그런데 새 버전으로 바뀌는 그 몇 초 동안 누군가 메시지를 보냈다면, 그 사람은 오류를 봤을까요. 그리고 바뀐 뒤에도 방금까지 나눈 대화가 그대로 남아 있을까요. 오늘은 서비스를 끄지 않고 새 버전으로 바꾸는 무중단 배포가 어떤 원리로 돌아가는지 배우고, 그 원리의 중심에 있는 로드밸런서를 내 컴퓨터에 Nginx로 직접 만들어 에이전트 서비스 두 벌 사이를 전환해 봅니다. 마지막으로 대화를 컨테이너 밖 데이터베이스로 옮겨, 어느 쪽으로 전환해도 대화가 이어지게 만듭니다.

주관식 퀴즈

Q. [시작 질문] 에이전트 서비스가 새 버전으로 바뀌는 순간, 사용자에게는 무슨 일이 일어날 수 있을까요? 오류, 끊김, 대화 내용 같은 것들 가운데 떠오르는 대로 적고, 그렇게 생각한 이유도 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. 가용성, 단일 장애점, 이중화, 무상태가 무엇이고 고가용성 서비스에서 각각 무엇을 막는지 설명할 수 있습니다.
2. 롤링, 블루그린, 카나리 배포의 차이와, 무중단 배포에서 헬스체크, 그레이스풀 셧다운, DB 호환성이 필요한 이유를 설명할 수 있습니다.
3. 로드밸런서가 서버를 고르고 고장 난 서버를 알아채는 방식과, Nginx가 리버스 프록시와 로드밸런서로 하는 일을 설정 파일로 읽을 수 있습니다.
4. 내 컴퓨터에서 Nginx와 에이전트 서비스 두 벌을 컨테이너로 띄우고, 설정 한 줄로 블루그린 전환과 롤백을 할 수 있습니다.

5. 전환하는 동안과 서버 하나가 멈췄을 때의 요청 실패를 측정하고, 그 결과를 Nginx 설정과 연결해 설명할 수 있습니다.
6. Supabase에 데이터와 익명 로그인을 붙여, 어느 쪽으로 전환해도 대화가 이어지는 서비스로 만들 수 있습니다.

Chapter02

가용성을 숫자로 말하기

가용성(availability)은 서비스를 쓸 수 있었던 시간의 비율입니다. 1년 내내 한 번도 멈추지 않았다면 100%입니다. 가용성이 아주 높아 거의 멈추지 않는 것을 고가용성(high availability)이라고 하고, 업계에서는 가용성을 9가 몇 개인지로 말합니다.

가용성	부르는 말	1년 중 멈춰도 되는 시간
99%	투 나인	약 3.65일
99.9%	쓰리 나인	약 8.8시간
99.99%	포 나인	약 53분
99.999%	파이브 나인	약 5분

99%는 높아 보이지만 1년에 사흘 넘게 멈춘다는 뜻입니다. 9가 하나 늘 때마다 멈춰도 되는 시간은 10분의 1로 줄고, 파이브 나인은 담당자가 알림을 받고 노트북을 켜는 사이에 지나갈 만큼 짧습니다. 그래서 고가용성의 핵심은 사람이 고치기 전에 시스템이 스스로 버티게 만드는 것입니다. 9를 하나 더 얻을 때마다 서버 수와 구조의 복잡도가 크게 늘기 때문에, 서비스마다 목표로 잡는 9의 개수는 다릅니다.

단일 장애점과 이중화

단일 장애점(SPOF, single point of failure)은 그것 하나만 고장 나도 전체가 멈추는 부분입니다. 서버가 한 대뿐이라면 그 서버가 단일 장애점입니다. 같은 일을 하는 서버나 장치를 하나 더 두어 단일 장애점을 없애는 것을 이중화(redundancy)라고 합니다. 두 대를 같은 데이터센터에 두면 정전이나 화재 때 함께 멈추므로, 중요한 서비스는 여러 데이터센터나 지역에 나눠 둡니다. 2022년 10월 판교 데이터센터 화재 때 카카오 서비스가 오래 멈춘 원인 가운데 하나도, 예비 서버로 넘길 때 쓰는 관리 도구를 그 데이터센터 안에서만 이중화해 둔 것이었습니다.

고가용성 기법은 복잡해 보여도 세 단계로 정리됩니다. 예비를 두는 이중화, 고장을 알아채는 장애 감지, 예비로 자동으로 넘기는 장애 조치(failover)입니다. 이중화에는 한 대가 일하고 한 대는 기다리다가 넘겨받는 액티브-스탠바이(active-standby)와, 모든 서버가 같이

일하다가 한 대가 멈추면 나머지가 그 몫까지 맡는 액티브-액티브(active-active) 방식이 있습니다.



그림: 서버 한 대, 액티브-스탠바이, 액티브-액티브에서 서버 하나가 고장 났을 때 요청이 가는 곳

무상태 서버

서버를 여러 대 두면 같은 사용자의 요청이 매번 다른 서버로 갈 수 있습니다. 로그인 정보나 대화 기록을 서버 A의 메모리에만 저장했는데 다음 요청이 서버 B로 가면, 사용자는 로그아웃되었거나 대화가 사라진 것처럼 봅니다. 그래서 이런 정보는 서버 밖의 공용 저장소(데이터베이스 등)에 두고, 서버는 어떤 요청이 와도 똑같이 처리하게 만듭니다. 이렇게 요청 처리에 필요한 상태를 자기 안에 두지 않는 서버를 무상태(stateless) 서버라고 합니다. 무상태여야 서버를 마음대로 늘리고, 빼고, 새 버전으로 바꿀 수 있습니다.

지금 에이전트 서비스는 무상태가 아닙니다. 캐릭터, 대화, 기억을 인스턴스 메모리에 두고 있어서, 새 리비전이 뜨면 그동안의 대화가 사라집니다. 이 차이를 섹션 5에서 직접 보고, 섹션 6에서 데이터베이스로 옮겨 해결합니다.

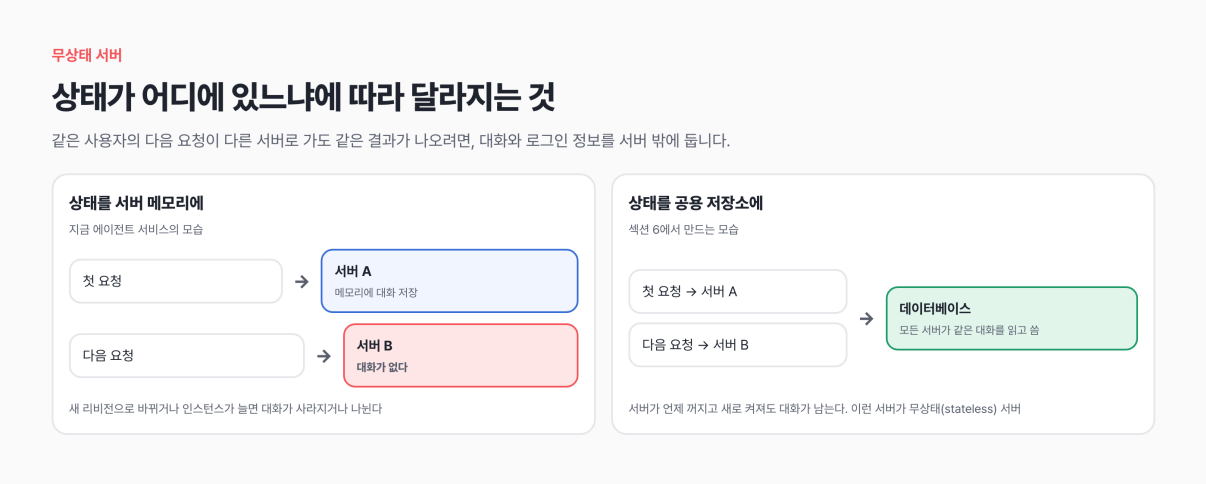


그림: 상태를 서버 메모리에 둘 때와 공용 저장소에 둘 때, 다음 요청이 다른 서버로 가면 생기는 차이

데이터베이스는 기억 그 자체라서 여러 대에 두려면 내용이 같아야 합니다. 가장 흔한 방법은 쓰기는 마스터(primary) 한 곳에서만 받고, 바뀐 내용을 여러 복제본(replica)에 계속 복사해 읽기를 나눠 맡기는 복제(replication)입니다. 마스터가 멈추면 가장 최신인 복제본을 새 마스터로 올립니다. 복제는 백업이 아니라서 실수로 지운 데이터도 곧바로 복제되므로, 되돌리려면 백업을 따로 둡니다. 이 과정에서 쓰는 Supabase는 이런 데이터베이스 운영을 대신 맡는 관리형 서비스입니다.

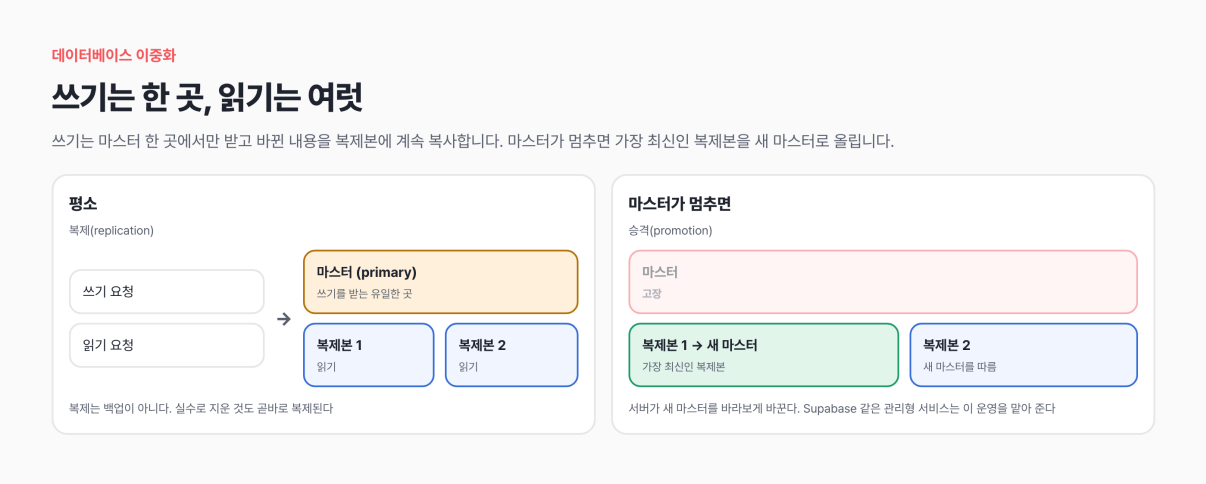


그림: 마스터 한 곳이 쓰기를 받아 복제본에 복사하고, 마스터가 멈추면 복제본 하나를 새 마스터로 올리는 흐름

무중단 배포와 세 가지 방식

예전에는 새 버전을 올릴 때 서버를 끄고 "점검 중" 화면을 띄운 뒤 새 코드를 올리고 다시 켰습니다. 하루에 여러 번 배포하는 서비스라면 이렇게 할 수 없고, 사용자에게는 업데이트 때

문에 멈추는 것도 고장으로 멈추는 것과 똑같습니다. 그래서 이전 버전을 끄지 않은 채 새 버전을 띄우고, 요청을 새 버전으로 옮긴 다음에 이전 버전을 끕니다. 이 방식을 무중단 배포(zero-downtime deployment)라고 하고, 모든 방법이 서버 여러 대와 그 앞에서 요청을 나눠 주는 로드밸런서를 바탕으로 합니다.

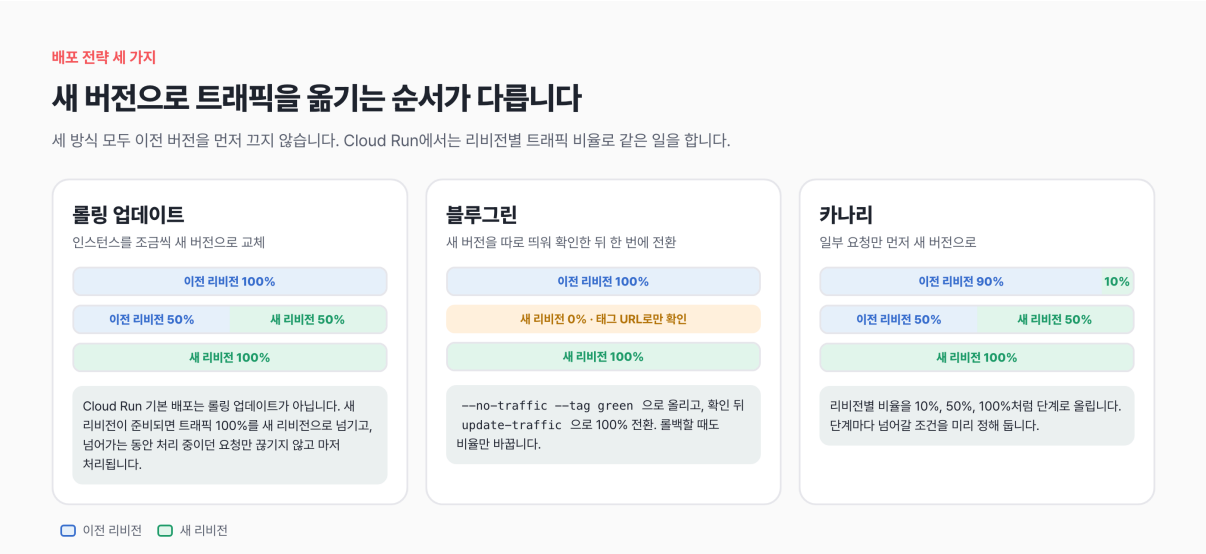


그림: 롤링, 블루그린, 카나리 배포를 버전별 트래픽 비율로 나타낸 그림

롤링 배포(rolling update)는 서버를 한 대씩 돌아가며 새 버전으로 바꿉니다. 서버를 더 두지 않아도 되지만, 바꾸는 동안 이전 버전과 새 버전이 함께 요청을 받습니다. 블루그린 배포(blue-green deployment)는 서버 묶음을 두 벌 준비해, 지금 요청을 받는 쪽(블루)은 그대로 두고 다른 쪽(그린)에 새 버전을 다 올린 뒤 로드밸런서의 방향을 한 번에 바꿉니다. 문제가 생기면 다시 블루로 돌리면 되고, 이렇게 이전 버전으로 되돌리는 것을 롤백(rollback)이라고 합니다. 가장 깔끔하지만 서버가 두 배 필요합니다. 카나리 배포(canary deployment)는 새 버전을 전체 요청의 1%나 10%에만 먼저 보내 보고, 지표가 괜찮으면 비율을 올립니다. 이름은 광부들이 유독가스를 먼저 알아채려고 카나리아를 데리고 갱도에 들어간 데서 왔습니다.

움직여 보는 이중화, 무상태, 배포 방식

아래 시각화에서 점 하나는 요청 하나이고, 빨간 점은 실패했거나 오류를 받은 요청입니다. ① 탭에서는 "서버 한 대", "액티브-스탠바이", "액티브-액티브"를 차례로 고르고 서버 1을 고장 내 봅니다. 로드밸런서가 고장을 알아채는 2초 동안 실패가 나고, 그 뒤 성공 비율이 어떻게 달라지는지 봅니다. ② 탭에서는 대화를 서버 메모리에 둘 때와 공용 저장소에 둘 때, 요청이 서버 A와 B를 오가면서 앞의 대화가 몇 개 보이는지 견줍니다. ③ 탭에서는 "새 버전에 오류 있음"을 켜고 롤링, 블루그린, 카나리로 한 번씩 배포해, 오류를 본 요청 수와 롤백했을 때 멈추는 시점을 비교합니다.

바꿀 때 지켜야 할 세 가지

어느 방식이든 새 버전으로 바꿀 때 지켜야 할 것이 세 가지 있습니다.

조건	하는 일	지키지 않으면
헬스체크(health check)	새 서버가 요청을 받을 준비가 됐는지 먼저 물어보고, 정상이라고 답해야 요청을 보냄	켜지는 중인 서버로 요청이 가서 오류가 남
그레이스풀 셧다운 (graceful shutdown)	이전 서버를 끌 때 처리 중이던 요청을 끝까지 마친 뒤 끔	답을 보내던 도중에 연결이 끊김
DB 호환성	이전 버전과 새 버전이 같은 데이터베이스를 함께 써도 둘 다 동작하게 함	새 버전이 테이블 구조를 바꾸면 전환 중이나 롤백 뒤에 이전 버전이 깨짐

마지막 줄이 가장 까다롭습니다. 서버는 여러 대지만 데이터베이스는 보통 하나를 함께 쓰므로, 새 버전이 열 이름을 바꾸면 전환하는 동안 요청을 받는 이전 버전은 그 열을 찾지 못하고, 롤백해도 같은 일이 생깁니다. 그래서 테이블 구조를 바꾸는 스키마 마이그레이션(schema migration)은 새 열을 먼저 추가하고, 두 열을 함께 쓰는 버전을 배포하고, 이전 버전이 다 내려간 뒤에 옛 열을 지우는 식으로 여러 번에 나눕니다. 이 순서를 확장-축소 패턴(expand and contract)이라고 합니다.

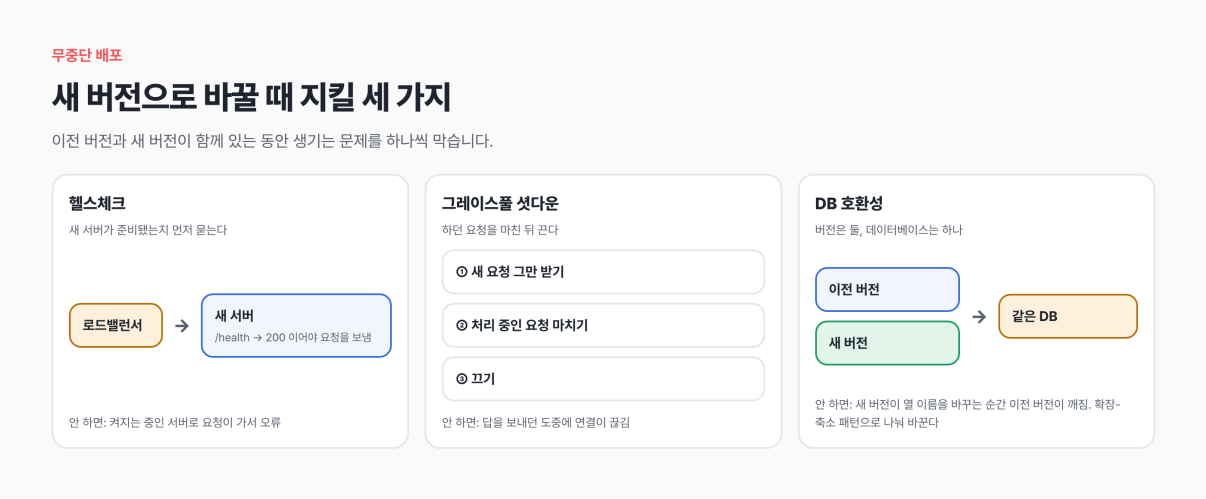


그림: 새 버전으로 바꿀 때 지킬 헬스체크, 그레이스풀 셧다운, DB 호환성

객관식 퀴즈

Q. 로그인 정보를 서버 메모리에만 저장하는 서비스를 서버 두 대와 로드밸런서로 운영하고 있습니다. 사용자가 새로고침할 때마다 로그아웃되는 일이 생긴다면, 가장 알맞은 해결은 무엇인가요?

- 1. 서버를 한 대로 줄인다
- 2. 로그인 정보를 서버 밖의 공용 저장소에 두어 서버를 무상태로 만든다
- 3. 로드밸런서의 헬스체크 간격을 줄인다
- 4. 블루그린 배포로 바꾼다**제출**

- •

  1. 서버를 한 대로 줄인다

- •

  2. 로그인 정보를 서버 밖의 공용 저장소에 두어 서버를 무상태로 만든다

- •

  3. 로드밸런서의 헬스체크 간격을 줄인다

- •

  4. 블루그린 배포로 바꾼다

📚 고가용성과 무중단 배포 참고 자료

- Embracing Risk (Site Reliability Engineering): 가용성 목표를 숫자로 정하고 멈춰도 되는 시간을 계산하는 방법을 볼 수 있습니다. 영어 문서입니다.
- Container runtime contract: Cloud Run이 인스턴스를 끌 때 종료 신호를 보내고 기다리는 시간처럼, 클라우드가 그레이스풀 셧다운을 어떻게 다루는지 확인할 수 있습니다.

Chapter03

로드밸런서가 하는 일

사용자는 서비스 주소 하나만 압니다. 서버가 여러 대라면 그 주소로 온 요청을 어느 서버로 보낼지 정하는 장치가 입구에 있어야 합니다. 이 장치가 로드밸런서(load balancer)입니다. 사용자가 주소로 요청하면 그 주소는 서버가 아니라 로드밸런서를 가리키고, 로드밸런서가 요청을 뒤의 서버 하나에 넘긴 뒤 그 서버의 응답을 받아 사용자에게 돌려줍니다. 사용자 쪽에서는 서비스 하나와 주고받은 것이고, 뒤에 서버가 몇 대인지는 알 수 없습니다.

요청을 어느 서버로 보낼지는 정해 둔 규칙을 따릅니다.

규칙	보내는 방식	쓰는 때
라운드 로빈(round robin)	1번, 2번, 3번, 다시 1번처럼 순서대로	서버 성능이 비슷할 때. 가장 단순함

규칙	보내는 방식	쓰는 때
최소 연결(least connections)	지금 처리 중인 요청이 가장 적은 서버로	요청마다 처리 시간이 크게 다를 때. LLM 응답처럼 몇 초에서 수십 초까지 걸리는 요청
가중치(weighted)	정해 둔 비율대로. 예: 90 대 10	성능이 다른 서버를 섞을 때, 카나리 배포
IP 해시(IP hash)	같은 사용자는 늘 같은 서버로	서버가 사용자 상태를 기억할 때. 다만 그 서버가 바뀌면 상태도 사라짐

IP 해시처럼 같은 사용자를 같은 서버로 묶는 방식을 세션 고정(sticky session)이라고 합니다. 서버가 상태를 갖고 있을 때 급한 대로 쓰는 방법이지만, 그 서버가 멈추거나 새 버전으로 바뀌면 묶여 있던 사용자의 상태가 함께 사라지고, 한 서버에 사용자가 몰려도 다른 서버로 나눌 수 없습니다. 그래서 섹션 2의 무상태 서버를 먼저 만들고, 규칙은 라운드 로빈이나 최소 연결을 쓰는 것이 보통입니다.

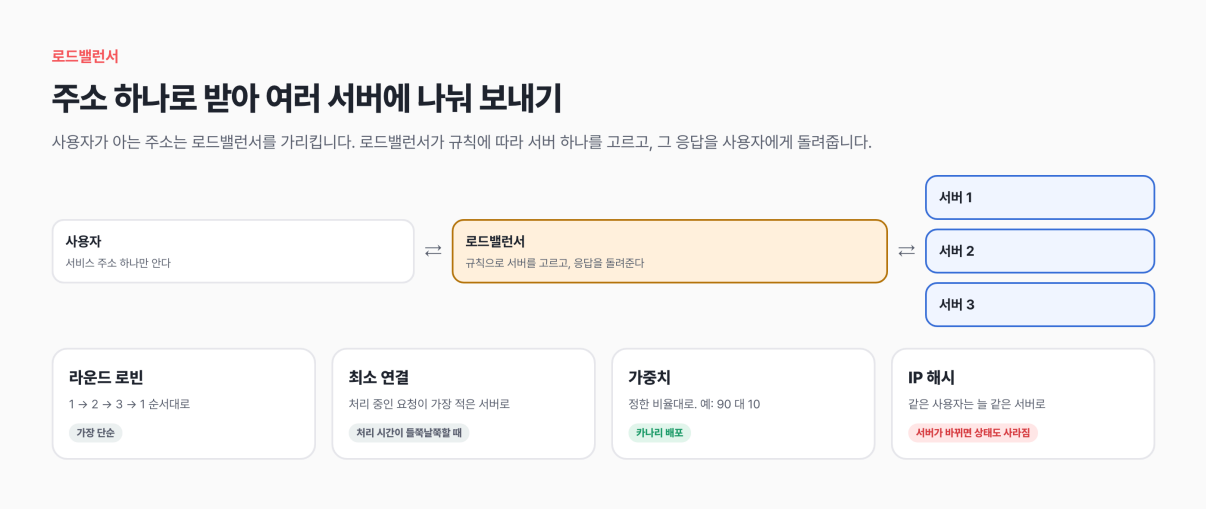


그림: 사용자 요청이 로드밸런서를 거쳐 서버 하나로 가는 흐름과 서버를 고르는 규칙 네 가지

헬스체크의 두 방식

로드밸런서가 고장 난 서버를 알아채는 방법은 두 가지입니다. 능동 헬스체크(active health check)는 로드밸런서가 몇 초마다 서버에 `/health` 같은 주소로 먼저 물어보고, 정해 둔 횟수만큼 답이 없으면 그 서버를 뺍니다. 사용자 요청이 고장 난 서버에 닿기 전에 미리 빼므로 사용자가 겪는 실패가 적습니다. 수동 헬스체크(passive health check)는 따로 묻지 않고, 실제 사용자 요청을 보냈다가 실패하면 그때 그 서버를 잠시 뺍니다. 설정할 것이 적지만, 고장을 알아채는 데 쓰인 첫 요청은 실제 사용자의 요청입니다.

	능동 헬스체크	수동 헬스체크
알아채는 방법	로드밸런서가 주기적으로 먼저 물어봄	실제 요청이 실패한 것을 보고 앎
사용자가 겪는 실패	확인 주기 사이에 간 요청 정도	알아채는 데 쓰인 요청과, 그 요청이 실패하기까지 기다린 시간
지원하는 곳	HAProxy, 클라우드 로드밸런서, 유료판 NGINX Plus	무료 Nginx를 포함한 대부분

무료 Nginx는 수동 헬스체크만 합니다. 한 서버로 보낸 요청이 실패하면(`max_fails`, 기본 1번) 그 서버를 일정 시간(`fail_timeout`, 기본 10초) 동안 빼고, 실패한 요청은 목록의 다른 서버로 다시 보냅니다. 문제는 실패라고 판단하기까지 걸리는 시간입니다. 서버가 아예 응답하지 않으면 Nginx는 연결을 기다리는 시간(`proxy_connect_timeout`, 기본 60초)이 지나야 실패로 보고 다른 서버로 넘기므로, 그 사이 요청을 보낸 사용자는 그만큼 기다립니다. 섹션 5에서 이 시간을 직접 재 보고 줄여 봅니다.

드레이닝과 로드밸런서 이중화

무중단 배포에서는 헬스체크를 반대 방향으로도 씁니다. 서버 한 대를 바꿀 때는 먼저 로드밸런서에서 그 서버로 새 요청을 보내지 않게 빼고, 처리 중이던 요청이 다 끝나면 끄고 업데이트합니다. 다시 켜서 헬스체크를 통과하면 다시 넣습니다. 이렇게 새 요청은 끊고 하던 요청은 마치게 하는 것을 드레이닝(draining)이라고 합니다.

로드밸런서는 모든 요청이 지나가는 곳이라 그 자체가 단일 장애점이 될 수 있습니다. 그래서 로드밸런서도 두 대를 액티브-스탠바이로 두고, 가상 IP(virtual IP)라는 주소 하나를 함께 씁니다. 평소에는 주 로드밸런서가 그 주소를 갖고 있다가, 멈추면 예비 로드밸런서가 넘겨받습니다. 사용자는 같은 주소로 접속하므로 바뀐 것을 알지 못하고, 클라우드의 로드밸런서는 이 일을 대신 해 줍니다.

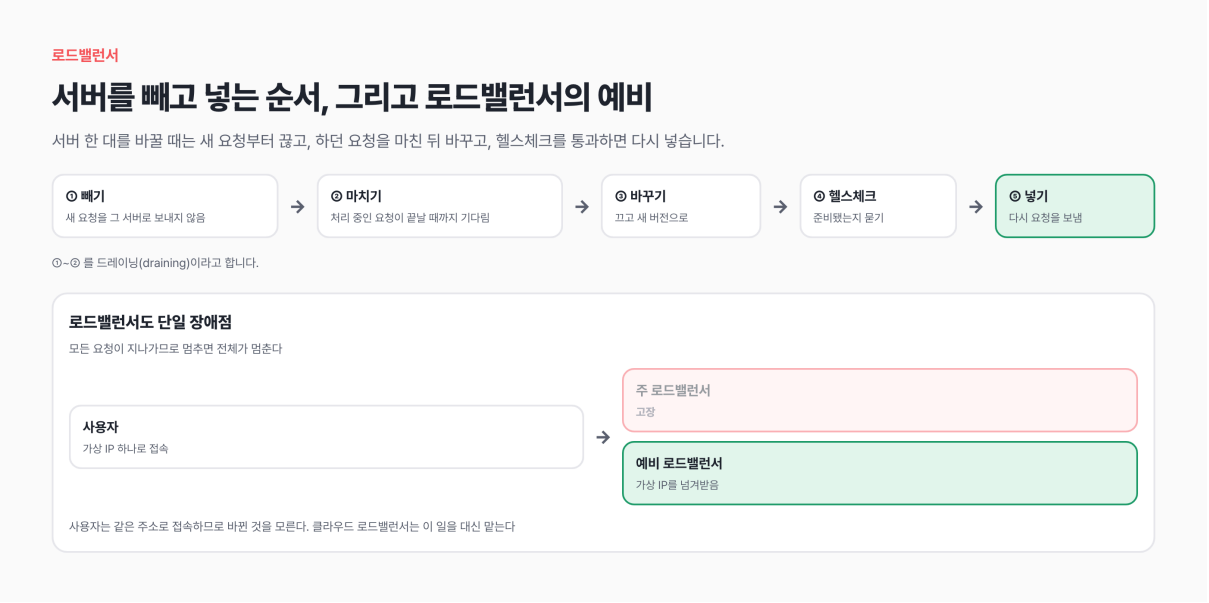


그림: 서버 한 대를 빼고 바꾸고 다시 넣는 순서와 가상 IP로 이중화한 로드밸런서

로드밸런서의 종류

종류	예	특징
소프트웨어	Nginx, HAProxy	서버 한 대에 설치하면 로드밸런서가 됨. 오늘 실습에서 씀
클라우드 서비스	AWS ELB, Google Cloud Load Balancing	빌려 씀. 로드밸런서 자체의 이중화와 능동 헬스체크를 클라우드가 맡음. Cloud Run도 서비스 안에 이 기능을 품고 있음
하드웨어	전용 장비	비싸서 요즘은 큰 회사에서 주로 씀

실제 서비스에서는 맨 앞을 클라우드 로드밸런서가 받고 그 뒤에 Nginx를 함께 두는 경우도 많습니다. 무중단 배포의 세 방식도 결국 로드밸런서를 조작하는 일입니다. 롤링은 로드밸런서에서 서버를 한 대씩 빼고 넣기를 반복하고, 블루그린은 로드밸런서가 가리키는 서버 묶음을 통째로 바꾸고, 카나리는 새 서버로 보내는 가중치를 조금씩 올립니다. DNS 주소를 바꾸는 방법도 있지만, 사용자 컴퓨터가 옛 주소를 한동안 기억해서 바로 바뀌지 않습니다.

객관식 퀴즈

Q. 무료 Nginx 뒤에 서버 두 대를 두었는데, 한 대의 전원이 갑자기 꺼졌습니다. 기본 설정이라면 사용자에게 어떤 일이 생길 가능성이 가장 높을까요?

- 1. Nginx가 몇 초마다 먼저 확인하므로 사용자는 아무것도 겪지 않는다
- 2. 꺼진 서버로 간 요청을 보낸 사용자가 연결을 기다리는 시간만큼 기다리거나 실패를 본

뒤, 그다음 요청부터는 다른 서버로 간다
• 3. Nginx가 멈추고 모든 요청이 실패한다
• 4. 꺼진 서버가 다시 켜질 때까지 모든 요청의 절반이 계속 실패한다**제출**

- •

  1. Nginx가 몇 초마다 먼저 확인하므로 사용자는 아무것도 겪지 않는다

- •

  2. 꺼진 서버로 간 요청을 보낸 사용자가 연결을 기다리는 시간만큼 기다리거나 실패를 본 뒤, 그다음 요청부터는 다른 서버로 간다

- •

  3. Nginx가 멈추고 모든 요청이 실패한다

- •

  4. 꺼진 서버가 다시 켜질 때까지 모든 요청의 절반이 계속 실패한다

Nginx

Nginx(엔진엑스)는 웹 서버 앞에서 요청을 받는 프로그램으로, 2004년에 동시 접속이 많을 때 느려지던 기존 웹 서버의 문제를 풀려고 만들어졌습니다. 지금은 전 세계 웹사이트의 30% 정도가 씁니다(W3Techs, 2026년 10월 기준). Nginx가 하는 일은 크게 세 가지입니다.

역할	하는 일
웹 서버	HTML, 이미지, CSS 같은 파일을 그대로 꺼내 응답
리버스 프록시(reverse proxy)	요청을 받아 뒤에 있는 애플리케이션 서버에 넘기고, 응답을 받아 사용자에게 돌려줌. 애플리케이션 서버는 바깥에 직접 드러나지 않음
로드밸런서	리버스 프록시를 넓힌 것. 뒤의 서버가 여러 대면 위 표의 규칙으로 나눠 보냄

포워드 프록시(forward proxy)가 사용자 쪽에 서서 바깥으로 나가는 요청을 대신 보낸다면, 리버스 프록시는 서버 쪽에 서서 바깥에서 들어오는 요청을 대신 받습니다. 방향이 반대라서 리버스라고 부릅니다.

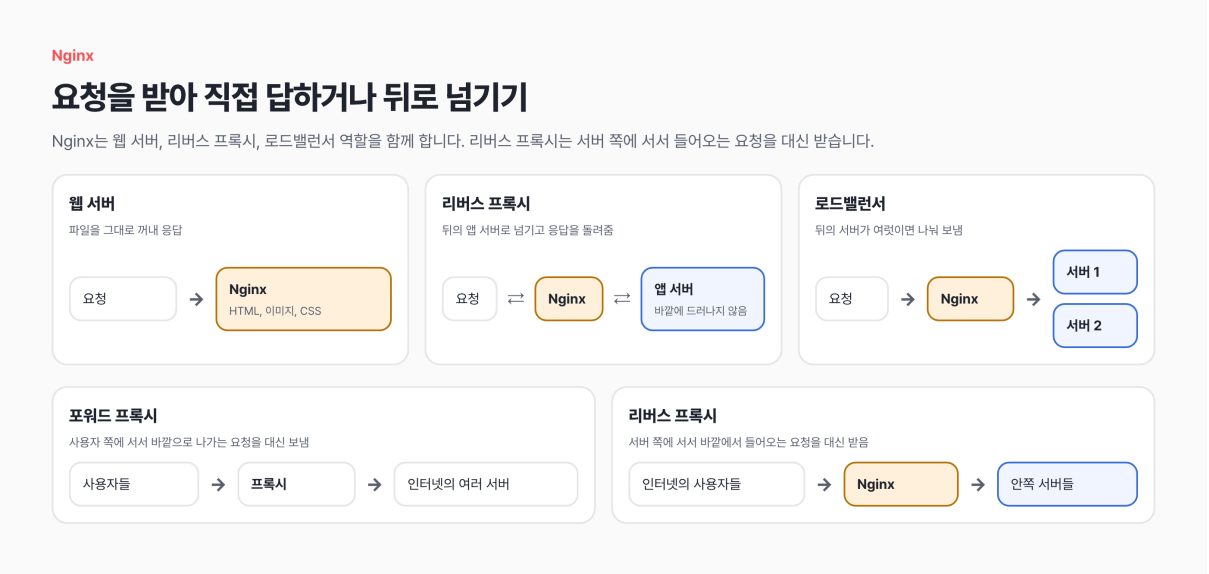


그림: Nginx의 세 역할과 포워드 프록시, 리버스 프록시의 방향 차이

Nginx가 많은 연결을 처리하는 방식

예전 웹 서버는 연결 하나마다 처리 단위(프로세스나 스레드)를 하나씩 붙였습니다. 접속이 1만 개면 처리 단위도 1만 개가 필요했고, 각각 메모리를 차지해서 금방 한계에 닿았습니다. Nginx는 작업 프로세스(worker process) 몇 개가 수천 개의 연결을 함께 맡습니다. 한 연결이 뒤의 서버 응답을 기다리는 동안 그 연결에 묶여 있지 않고 다른 연결을 처리하다가, 응답이 도착했다는 알림(이벤트)이 오면 돌아와 이어서 처리합니다. 이 방식을 이벤트 기반(event-driven) 비동기 처리라고 하고, 지금은 다른 웹 서버도 대부분 같은 방식을 씁니다.

작업 프로세스는 보통 CPU 코어 수만큼 띄웁니다. 작업 프로세스는 서로 메모리를 나눠 쓰지 않으므로, 라운드 로빈의 "다음은 몇 번 서버" 같은 순서도 프로세스마다 따로 셉니다. 요청이 많을 때는 결과적으로 고르게 나뉘지만, 요청을 한 번에 하나씩만 보내는 실습에서는 매번 다른 프로세스가 받아 같은 서버로만 가는 것처럼 보일 수 있습니다. upstream 블록에 `zone` 을 적으면 작업 프로세스들이 공유 메모리에서 순서와 실패 기록을 함께 셉니다.

Nginx 설정 파일 읽기

Nginx는 설정 파일로 동작을 정합니다. 설정은 중괄호로 묶은 블록으로 되어 있고, `server` 블록이 "어느 포트로 들어온 요청을 받을지", 그 안의 `location` 블록이 "어떤 경로의 요청을 어떻게 처리할지", `upstream` 블록이 "넘겨줄 서버 목록"을 정합니다. 로드밸런서로 쓰는 설정은 아래 정도입니다.

```
upstream my_app {
    zone my_app 64k;
```

```
    server 10.0.0.1:8080 weight=3;
    server 10.0.0.2:8080;
    server 10.0.0.3:8080 backup;
}

server {
    listen 80;
    location / {
        proxy_pass http://my_app;
        proxy_connect_timeout 2s;
    }
}
```

위 설정은 80번 포트로 들어온 모든 경로(`/`)의 요청을 `my_app` 목록으로 넘깁니다. 목록의 서버 줄 뒤에는 그 서버를 어떻게 다룰지 덧붙일 수 있습니다.

설정	뜻
`weight=3`	다른 서버보다 3배 많이 보냄. 적지 않으면 1
`backup`	다른 서버가 모두 빠졌을 때만 보냄
`max_fails=1 fail_timeout=10s`	10초 안에 1번 실패하면 10초 동안 뺌. 적지 않으면 이 값
`zone my_app 64k`	작업 프로세스들이 순서와 실패 기록을 공유 메모리에서 함께 셈
`proxy_connect_timeout 2s`	뒤의 서버에 연결될 때까지 2초만 기다리고, 넘으면 실패로 보고 다른 서버로 보냄. 적지 않으면 60초

가중치가 3 대 1이면 요청 4개 가운데 3개가 첫 서버로 가므로, 이 비율을 9 대 1이나 99 대 1로 두면 카나리 배포가 됩니다. 블루그린은 목록에 한 줄만 두고, 전환할 때 그 줄을 다른 서버로 바꾸는 것입니다.

설정을 바꾸는 reload

설정을 바꾼 뒤에는 `nginx -t` 로 문법을 검사하고, `nginx -s reload` 로 다시 읽게 합니다. reload는 Nginx를 재시작하지 않고 설정만 바꿔 끼웁니다. 기존 작업 프로세스는 처리하던 요청을 마무리하고, 새 작업 프로세스가 새 설정으로 다음 요청부터 받으므로, 설정을 바꾸는 순간에도 Nginx 쪽에서 끊기는 요청이 없습니다. 문법 검사가 실패한 설정으로 reload하면 Nginx는 새 설정을 버리고 이전 설정으로 계속 돕니다. 그래도 검사를 먼저 하는 이유는 실패를 사람이 바로 보고 고치기 위해서입니다.

주의할 점이 두 가지 있습니다. 첫째, Nginx는 뒤의 서버를 관리하지 않습니다. 전환한 직후 이전 서버를 바로 꺼 버리면 그 서버가 처리하던 요청은 끊길 수 있고, 롤백할 곳도 사라집니다. 둘째, Nginx는 설정을 읽을 때 `blue` 같은 서버 이름을 IP 주소로 바꿔 기억해 둡니다. 뒤의 서버를 지우고 다시 띄워 IP가 바뀌면 Nginx는 옛 주소로 계속 보내므로, 다시 띄운 뒤에는 reload해서 이름을 다시 찾게 합니다.

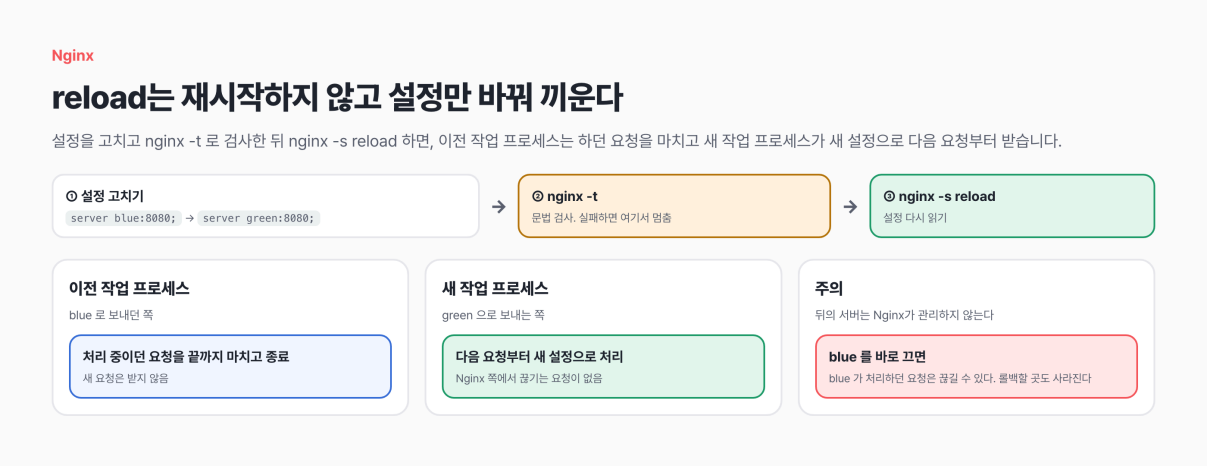


그림: 설정을 고치고 nginx -t 로 검사한 뒤 reload 할 때 이전 작업 프로세스와 새 작업 프로세스가 하는 일

움직여 보는 로드밸런서

지금까지 읽은 분배 규칙, 헬스체크와 드레이닝, reload를 아래 시각화로 확인합니다. ① 탭에서는 규칙을 바꿔 가며 요청이 어느 서버로 가는지와 서버마다 처리 중인 요청 수(막대)를 봅니다. 최소 연결은 처리 시간이 긴 요청이 섞여도 막대가 고르게 유지되고, IP 해시는 점 색(사용자)마다 늘 같은 서버로 갑니다. ② 탭에서는 능동 헬스체크와 수동 헬스체크를 각각 고르고 "서버 2 응답 없음"을 눌러 실패 수가 어떻게 쌓이는지 비교하고, "서버 2 드레이닝 후 교체"로 빼고 바꾸고 다시 넣는 순서를 봅니다. ③ 탭에서는 reload한 뒤 이전 작업 프로세스가 blue로 보낸 요청을 끝까지 받는 모습을 보고, 처음부터 다시 해서 reload 직후 blue를 바로 끄면 끊기는 요청이 생기는 것을 봅니다.

객관식 퀴즈

Q. Nginx 설정의 `upstream` 블록에 `zone` 줄과 함께 `server blue:8080;` 과 `server green:8080;` 두 줄을 적고 reload했습니다. 요청 확인 스크립트로 요청을 계속 보내면 어떻게 보일까요?

- 1. 항상 blue만 응답한다
- 2. blue와 green이 번갈아 응답한다
- 3. 항상 green만 응답한다
- 4. 설정 오류로 Nginx가 멈춘다**제출**

- •

  1. 항상 blue만 응답한다

- •

  2. blue와 green이 번갈아 응답한다

- •

  3. 항상 green만 응답한다

- •

  4. 설정 오류로 Nginx가 멈춘다

📚 로드밸런서와 Nginx 참고 자료

- NGINX: Using nginx as HTTP load balancer: upstream 블록, 라운드 로빈, 최소 연결, IP 해시, 가중치 설정을 공식 문서로 볼 수 있습니다.
- NGINX: Module ngx_http_upstream_module: `weight`, `backup`, `max_fails`, `fail_timeout`, `zone` 이 무엇을 하는지와 기본값이 나옵니다.
- NGINX: Controlling nginx: reload 신호를 받으면 작업 프로세스를 어떻게 바꿔 끼우는지 설명합니다.

Chapter04

시작 상태 맞추기

이 교안의 실습은 모두 내 컴퓨터의 Docker로 하고, Google Cloud는 쓰지 않습니다. 필요한 것은 Docker Desktop, 저장소 루트의 Dockerfile로 만든 에이전트 서비스 이미지 `friend-agent:local`, 요청 확인 스크립트를 돌릴 저장소의 가상환경입니다.

앞 실습을 이어서 하고 있다면 아래 확인 명령만 실행하고 넘어갑니다. 빠진 상태가 있으면 템플릿의 실습별 시작 상태 문서에서 "없으면" 칸의 항목을 표의 위 칸부터 차례로 실행합니다. 처음부터 모두 맞추면 한두 시간이 걸리므로, 빠진 항목이 많다면 수업 전에 맞춰 두거나 강사에게 먼저 알립니다.

필요한 상태	확인할 것	없으면
템플릿으로 만든 내 저장소와 가상환경	가상환경을 켜고 `python -c "import httpx"` 가 오류 없이 끝남	A의 3번, 저장소 README의 로컬 실행 준비
Docker Desktop과 저장소 루트의 `Dockerfile`, `.dockerignore`	`docker info` 가 오류 없이 나오고, 두 파일이 루트에 있음	F
에이전트 서비스 이미지 `friend-agent:local`	`docker images friend-agent:local` 에 한 줄이 나옴	저장소 루트에서 `docker build -t friend-agent:local .`

직접 실행할 명령: 시작 상태 확인

```
python -c "import httpx"
docker info
ls Dockerfile .dockerignore
docker images friend-agent:local
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 의 표를 보고, 이 저장소와 지금 gcloud 프로젝트가 '무중단 배포' 실습을 시작할 상태인지 확인해 줘.
확인 명령만 실행하고 아무것도 만들거나 바꾸지 마. 빠진 항목이 있으면 문서의 어느 항목(A~I)을 실행해야 하는지 알려 줘.
```

실습 구조

섹션 3에서 본 블루그린을 내 컴퓨터에서 그대로 만듭니다. 서버 두 벌 대신 에이전트 서비스 컨테이너 두 개(blue, green)를 띄우고, 그 앞에 Nginx 컨테이너 하나를 로드밸런서로 세웁니다. 세 컨테이너는 같은 Docker 네트워크에 두어 이름으로 서로 찾게 합니다. 같은 네트워크에 있는 컨테이너는 `blue:8080` 처럼 컨테이너 이름과 포트로 다른 컨테이너에 요청할 수 있습니다. 브라우저와 요청 확인 스크립트는 내 컴퓨터의 8080번 포트로 Nginx에만 요청하고, 어느 쪽으로 보낼지는 Nginx 설정의 `upstream` 한 줄이 정합니다.



그림: Docker 네트워크 안의 Nginx 컨테이너와 에이전트 서비스 컨테이너 두 개. upstream 한 줄이 활성 쪽을 정한다

두 컨테이너는 같은 이미지로 띄우고, 환경변수 두 개만 다르게 줍니다. `BANNER` 는 화면 위에 BLUE 또는 GREEN 배너를 보여 주고, `K_REVISION` 은 `/version` 응답의 `revision` 값이 됩니다. `K_REVISION` 은 Cloud Run이 리비전 이름을 알려 줄 때 쓰는 환경변수인데, 여기서는 로컬 컨테이너에 직접 넣어 어느 쪽이 응답했는지 구별하는 데 씁니다. LLM API는 부르지 않도록 페이크 LLM(`LLM_PROVIDER=fake`)으로 띄웁니다. 전환을 확인하는 데는 실제 답이 필요 없고, 키를 컨테이너에 넣지 않아도 됩니다.

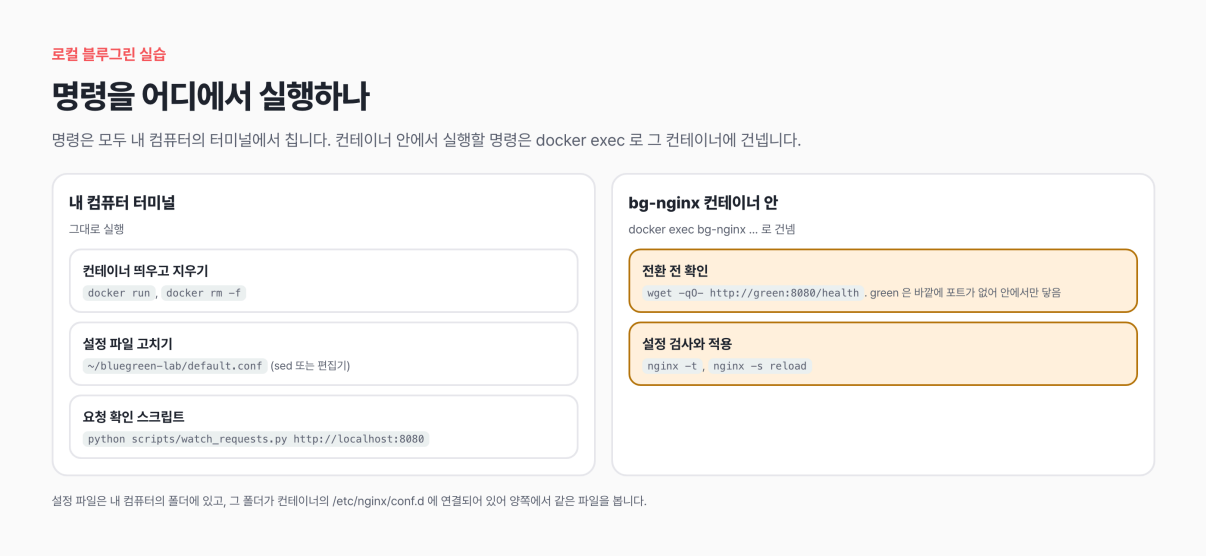


그림: 실습 명령 중 내 컴퓨터 터미널에서 그대로 치는 것과 docker exec 로 bg-nginx 컨테이너에 건네는 것

Windows에서 다른 명령

이 교안의 명령은 macOS와 Linux 터미널 기준입니다. `docker` 로 시작하는 명령은 Windows PowerShell에서도 그대로 쓰고, `-v "$HOME/bluegreen-lab:/etc/nginx/conf.d"` 도 PowerShell에서 내 홈 폴더로 바뀝니다. 가상환경은 `.venv\Scripts\Activate.ps1` 로 켭니다. 내 컴퓨터의 파일을 다루는 몇 가지 명령만 아래처럼 바꿉니다.

교안 명령	Windows PowerShell에서
`mkdir -p ~/bluegreen-lab`	`mkdir $HOME\bluegreen-lab`
`cat > ~/bluegreen-lab/default.conf <<'EOF'` 부터 `EOF` 까지	편집기(VS Code 등)로 `default.conf` 를 열어 그 사이의 내용을 붙여 넣고 저장
`sed -i '' 's/server blue:8080;/server green:8080;/' …`	편집기로 `default.conf` 의 서버 줄을 직접 고치고 저장
`ls Dockerfile .dockerignore`	`ls Dockerfile, .dockerignore`

교안 명령	Windows PowerShell에서
`docker logs blue 2>&1 | head -5`	`docker logs blue 2>&1 | Select-Object -First 5`

편집기로 저장한 `default.conf` 의 줄바꿈 형식이 Windows 방식(CRLF)이어도 Nginx는 그대로 읽습니다. 저장한 뒤에는 macOS와 같이 `docker exec bg-nginx nginx -t` 와 `docker exec bg-nginx nginx -s reload` 를 실행합니다.

설정 파일과 컨테이너 세 개 띄우기

실습용 폴더를 저장소 밖에 하나 만들고 그 안에 Nginx 설정 파일 `default.conf` 를 둡니다. 처음 설정은 섹션 3에서 읽은 모양에서 서버를 blue 한 줄만 둔 것이고, `zone` 줄은 섹션 5에서 서버를 두 줄로 늘릴 때 작업 프로세스들이 순서를 함께 세게 하려고 미리 넣어 둡니다. Nginx 컨테이너에는 이 파일이 아니라 폴더를 연결합니다. 편집기나 명령으로 파일을 고치면 파일이 새로 만들어지는 경우가 많은데, 파일 하나만 연결해 두면 컨테이너는 새 파일을 보지 못하기 때문입니다.

폴더를 연결하는 `-v` 옵션에는 홈 폴더를 `~` 대신 `$HOME` 으로 씁니다. `~` 를 따옴표 안에 넣으면 홈 폴더 경로로 바뀌지 않아, 설정 폴더가 연결되지 않은 채 Nginx가 기본 설정으로 뜹니다. 내 컴퓨터에서 다른 프로그램이 8080번 포트를 쓰고 있다면 Nginx 쪽 `-p` 의 앞 숫자를 다른 번호로 바꾸고, 이후 주소도 그 번호로 바꿉니다.

직접 실행할 명령: 설정 파일과 컨테이너 세 개 띄우기 (macOS, Linux)

```
# 실습 폴더와 Nginx 설정 파일. 처음에는 blue 로 보낸다
mkdir -p ~/bluegreen-lab
cat > ~/bluegreen-lab/default.conf <<'EOF'
upstream active_app {
    zone active_app 64k;
    server blue:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
    }
}
EOF

# 세 컨테이너가 이름으로 서로 찾는 Docker 네트워크
docker network create bg-lab

# 에이전트 서비스 두 벌 (같은 이미지, 환경변수만 다름)
docker run -d --name blue  --network bg-lab -e LLM_PROVIDER=fake -e K_REVISION=blue  -
e BANNER=BLUE  friend-agent:local
docker run -d --name green --network bg-lab -e LLM_PROVIDER=fake -e K_REVISION=green -
```

```
e BANNER=GREEN friend-agent:local

# 로드밸런서 역할의 Nginx. 내 컴퓨터 8080 → 컨테이너 80, 설정 폴더 연결
docker run -d --name bg-nginx --network bg-lab -p 8080:80 -v "$HOME/bluegreen-lab:/et
c/nginx/conf.d" nginx:1.27-alpine
docker ps
```

코딩 에이전트에게 맡길 때: 설정 파일과 컨테이너 세 개 띄우기

```
내 홈 폴더에 bluegreen-lab 폴더를 만들고 그 안에 Nginx 설정 파일 default.conf 를 아래 내용으로 만들
어 줘.
upstream active_app {
    zone active_app 64k;
    server blue:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
    }
}
그다음 Docker 네트워크 bg-lab 을 만들고, friend-agent:local 이미지로 컨테이너 blue 와 green 을
이 네트워크에 띄워 줘.
둘 다 LLM_PROVIDER=fake 로 하고, blue 는 K_REVISION=blue, BANNER=BLUE, green 은 K_REVISIO
N=green, BANNER=GREEN 을 넣어 줘. 포트는 열지 마.
마지막으로 nginx:1.27-alpine 이미지로 bg-nginx 컨테이너를 같은 네트워크에 띄우고, 내 컴퓨터 8080 을
컨테이너 80 에 연결하고, bluegreen-lab 폴더를 /etc/nginx/conf.d 에 연결해 줘. 폴더 경로는 $HOME
으로 써 줘.
```

Windows PowerShell에서는 위 "Windows에서 다른 명령" 표처럼 `default.conf` 를 편집기로 만들고, 나머지 `docker` 명령은 그대로 실행합니다. blue와 green에는 `-p` 를 붙이지 않았다는 점을 봅니다. 두 컨테이너는 내 컴퓨터에 포트를 열지 않아 바깥에서 직접 들어갈 수 없고, 같은 네트워크의 Nginx만 요청을 넘길 수 있습니다. 섹션 3에서 본 리버스 프록시의 모습입니다.

직접 확인할 곳: 세 컨테이너 확인

1. `docker ps` 에 `blue`, `green`, `bg-nginx` 세 줄이 보이고, `bg-nginx` 의 포트 칸에만 `8080->80` 이 있는지 봅니다.
2. `docker exec bg-nginx cat /etc/nginx/conf.d/default.conf` 로 Nginx가 보는 설정이 위에서 만든 내용인지 봅니다.
3. 브라우저로 `http://localhost:8080` 을 열어 화면 위에 BLUE 배너가 보이는지 봅니다. "Welcome to nginx!" 가 보이는데 `/version` 은 정상이라면, 설정 없이 Nginx를 먼저 띄웠을 때의 화면이 브라우저 캐시에 남은 것이므로 강력 새로고침(macOS `Cmd+Shift+R`, Windows `Ctrl+Shift+R`)을 합니다.

4. `http://localhost:8080/version` 을 열어 `revision` 이 `blue` 인지 봅니다.

```
jecha@Adminui-MacBookPro Aiffel % docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS                                       NAMES
b6199d1665fd   nginx:1.27-alpine    "/docker-entrypoint.…"   6 seconds ago    Up 5 seconds    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     bg-nginx
bec533847540   friend-agent:local   "sh -c 'uvicorn app.…"   10 minutes ago   Up 10 minutes                                               green
904894b1caec   friend-agent:local   "sh -c 'uvicorn app.…"   11 minutes ago   Up 11 minutes                                               blue
```

그림: docker ps에 blue, green, bg-nginx가 실행 중

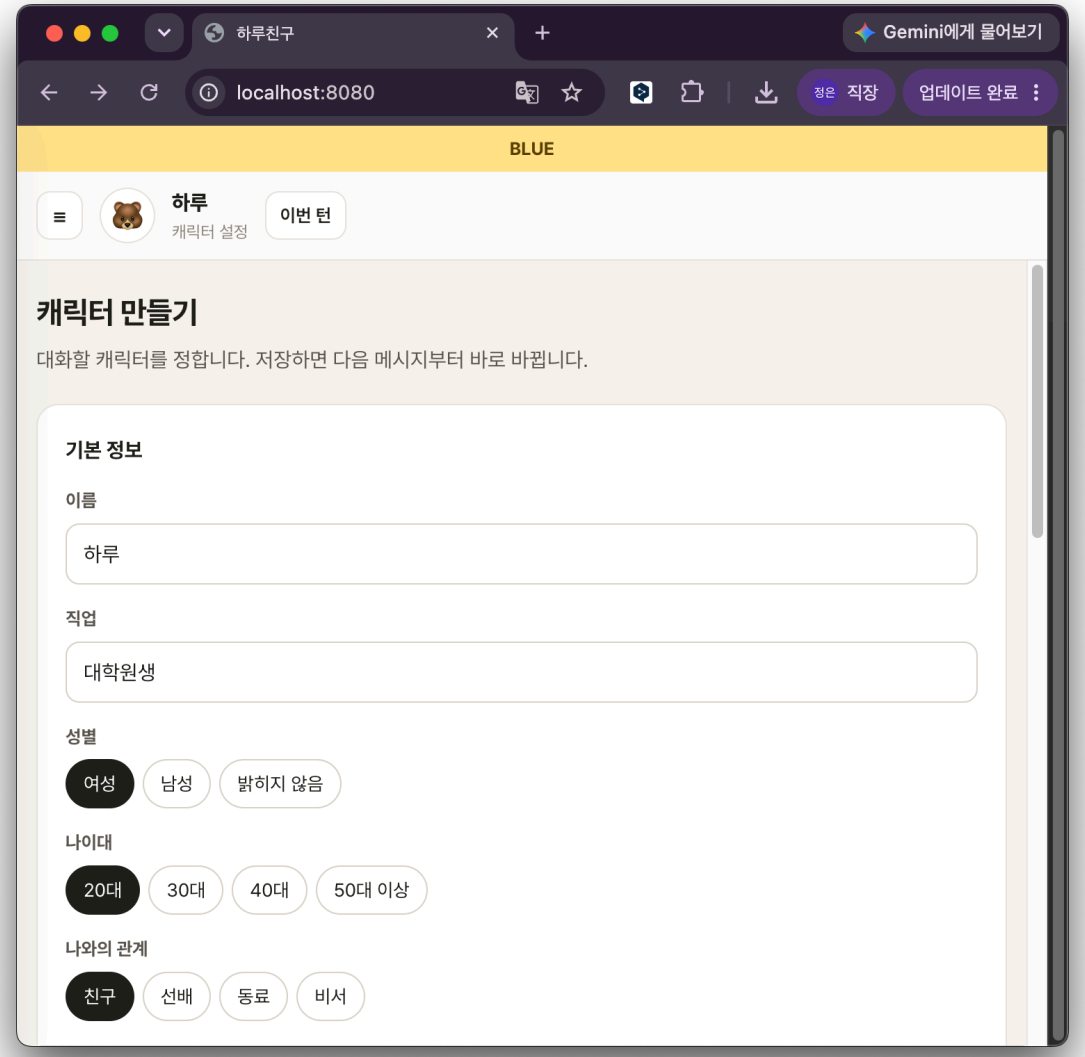


그림: 브라우저 화면 위에 BLUE 배너

Chapter05

요청 확인 스크립트

전환하는 동안 사용자가 오류를 봤는지는 새로고침 몇 번으로는 알 수 없습니다. 템플릿의 `scripts/watch_requests.py` 는 주소의 `/version` 에 0.2초마다 요청을 보내고, 응답 코드와 응답한 `revision` 을 기록합니다. 연결이 실패하거나 2초 안에 답이 없어도 실패로 셉니다. `--seconds` 로 정한 시간(최대 5분)이 지나거나 Ctrl+C로 멈추면, 전체 요청 수, 실패 수, `revision` 별 응답 수를 요약해 보여 줍니다. `/version` 은 LLM API를 부르지 않으므로 오래 돌려도 비용이 들지 않습니다.

터미널을 두 개 엽니다. 한쪽에서는 스크립트를 실행해 두고, 다른 쪽에서 전환 명령을 실행합니다.

실험을 먼저 눈으로 보기

이 섹션에서는 세 가지를 직접 해 봅니다. blue에서 green으로 전환했다가 롤백하고, 두 서버에 3 대 1로 나눠 보내고, 나눠 보내는 중에 green을 멈춥니다. 명령을 치기 전에 아래 시뮬레이터에서 각 실험이 무엇을 하는지 먼저 봅니다. 점 하나가 요청 확인 스크립트가 보낸 요청 하나이고, Nginx를 거쳐 파란 점은 blue, 초록 점은 green이 응답한 것입니다. 흰 점은 Nginx가 멈춘 서버에 연결되기를 기다리는 요청입니다. 오른쪽에는 그 순간의 설정 파일과 스크립트가 세는 숫자가 나옵니다.

① 탭에서 "green으로 전환"을 누르면 설정의 서버 줄이 바뀌고 다음 요청부터 초록 점이 됩니다. ② 탭에서는 파란 점 세 개마다 초록 점 하나가 나갑니다. ③ 탭에서 "docker stop green"을 누르면 green 차례의 요청이 흰 점으로 멈춰 있다가 2초가 지나 실패로 세어지고, `proxy_connect_timeout 1초` 를 누른 뒤 같은 일을 하면 흰 점이 1초 만에 blue로 넘어갑니다. 아래 실습에서 터미널에 찍히는 숫자가 시뮬레이터와 같은 모양인지 견주어 봅니다.

직접 실행할 명령: 요청 확인 스크립트 실행 (첫 번째 터미널)

```
# 저장소 루트, 가상환경을 켠 터미널에서
python scripts/watch_requests.py http://localhost:8080
```

green으로 전환하기

새 버전이 green에 올라갔다고 생각하고 전환합니다. 전환하기 전에 green이 정상인지 먼저 확인합니다. green은 바깥에 포트를 열지 않았으므로, 같은 네트워크 안에 있는 Nginx 컨테이너 안에서 green에게 직접 물어봅니다. 섹션 3에서 본 것처럼 무료 Nginx는 이 확인을 미리 해 주지 않습니다. 정상이면 설정 파일의 `upstream` 줄을 `green:8080` 으로 바꾸고, 문법을 검사한 뒤 reload합니다.

직접 실행할 명령: green 확인하고 전환하기 (두 번째 터미널, macOS)

```
# 1. 전환 전 확인: Nginx 컨테이너 안에서 green 의 /health 와 /version
docker exec bg-nginx wget -qO- http://green:8080/health
docker exec bg-nginx wget -qO- http://green:8080/version

# 2. upstream 줄을 blue 에서 green 으로 바꾼다 (Linux 는 sed -i 's/…/…/')
sed -i '' 's/server blue:8080;/server green:8080;/' ~/bluegreen-lab/default.conf

# 3. 문법 검사 뒤 reload
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

코딩 에이전트에게 맡길 때: green 확인하고 전환하기

```
bg-nginx 컨테이너 안에서 http://green:8080/health 와 /version 을 확인해 줘. 둘 다 정상이면
~/bluegreen-lab/default.conf 의 upstream 줄을 server green:8080; 으로 바꾸고, bg-nginx 에서
nginx -t 로 검사한 뒤 nginx -s reload 해 줘.
검사가 실패하면 reload 하지 말고 오류를 보여 줘. 다른 줄은 바꾸지 마.
```

Windows에서는 편집기로 `default.conf` 를 열어 `blue` 를 `green` 으로 직접 고치고 저장한 뒤 3번만 실행합니다. 실행 전에는 바뀌는 줄이 `upstream` 안의 한 줄뿐인지, `nginx -t` 가 성공한 다음에만 reload하는지 봅니다.

결과 해석: 끊기지 않고 바뀐 응답

브라우저를 새로고침하면 배너가 GREEN으로 바뀌고, `/version` 의 `revision` 이 `green` 이 됩니다. 스크립트를 멈추면 요약에 `blue` 와 `green` 의 응답 수가 함께 찍히고 실패는 0건입니다. reload는 Nginx를 재시작하지 않고, 이전 작업 프로세스가 하던 요청을 마치는 동안 새 작업 프로세스가 다음 요청부터 green으로 보내기 때문입니다. 응답한 쪽이 바뀐 시각은 스크립트 기록에서 `revision` 이 바뀐 줄로 찾을 수 있습니다. 실패가 있다면 그 시각을 reload한 시각과 맞춰 보고, 배포와 상관없이 드문드문 났다면 내 컴퓨터의 일시적인 문제일 수 있으니 한 번 더 측정합니다.

```
(.venv) jecha@Adminui-MacBookPro friend-agent % python scripts/watch_requests.py http://localhost:8080
http://localhost:8080/version 에 0.2초마다 요청합니다. 최대 300초 뒤 멈춥니다. 멈추려면 Ctrl+C.
16:55:58.603  응답 리비전: blue
16:58:25.020  응답 리비전: green
^C
── 요약 ──
전체 요청 1013건, 실패 0건, 걸린 시간 215초
  blue: 689건 (68%)
  green: 324건 (32%)
실패가 있으면 그 시각을 배포 단계(콘솔, Actions 로그의 시각)와 맞춰 본다. 내 네트워크 문제도 실패로 잡힐
수 있다.
```

그림: 요청 확인 스크립트를 실행한 터미널: 요약에 blue 와 green 의 응답 수가 함께 있고 실패 0건인 모습

롤백

이제 green에 문제가 있다고 가정하고 blue로 돌아갑니다. 스크립트를 다시 켜 두고, 설정의 같은 줄을 `blue:8080` 으로 되돌린 뒤 reload합니다. blue는 전환한 뒤에도 끄지 않고 그대로 켜 두었으므로, 몇 초 만에 원래대로 돌아갑니다. 블루그린 롤백이 빠른 이유는 이전 버전을 바로 끄지 않고 남겨 두기 때문이고, 반대로 이전 버전을 일찍 끄면 롤백하기 전에 다시 띄워야 합니다.

직접 실행할 명령: blue로 롤백하기 (macOS)

```
sed -i '' 's/server green:8080;/server blue:8080;/' ~/bluegreen-lab/default.conf
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

두 서버에 나눠 보내기

블루그린은 upstream에 한 줄만 두었습니다. 이번에는 섹션 3의 가중치 설정으로 두 줄을 두고, blue에 3, green에 1의 비율로 나눠 보냅니다. 새 버전에 요청 일부만 보내는 카나리 배포가 로드밸런서에서는 이 설정 한 줄로 됩니다. 이번에는 바꿀 곳이 여러 줄이라 설정 파일 전체를 다시 씁니다. 시뮬레이터의 ② 탭과 같은 설정입니다.

직접 실행할 명령: blue 3 대 green 1로 나눠 보내기 (macOS, Linux)

```
cat > ~/bluegreen-lab/default.conf <<'EOF'
upstream active_app {
    zone active_app 64k;
    server blue:8080 weight=3;
    server green:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
    }
}
EOF
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

```
# 첫 번째 터미널에서 30초 동안 측정
python scripts/watch_requests.py http://localhost:8080 --seconds 30
```

Windows에서는 편집기로 `default.conf` 를 열어 위의 `cat` 과 `EOF` 사이 내용으로 바꿔 저장한 뒤, 나머지 줄을 실행합니다.

결과 해석: 가중치대로 나뉜 응답

요약의 `revision` 별 응답 수가 blue 약 75%, green 약 25%로 나뉩니다. 요청 4개마다 3개가 blue로 가도록 Nginx가 순서를 세기 때문이고, `zone` 줄 덕분에 여러 작업 프로세스가 같은 순서를 함께 셉니다. 비율이 정확히 3 대 1이 아니라 조금 어긋나는 것은 측정 시작과 끝이 순서 중간에 걸리기 때문입니다. 브라우저에서 새로고침을 여러 번 하면 BLUE와 GREEN 배너가 섞여 보이는데, 지금 에이전트 서비스라면 새로고침마다 다른 컨테이너의 메모리를 보게 되므로 대화도 뒤섞입니다. 이 문제는 아래 "전환하면 사라지는 대화"에서 따로 봅니다.

서버 하나가 멈췄을 때

두 줄로 나눠 보내는 상태에서 green이 갑자기 멈추면 어떻게 될까요. 섹션 3에서 무료 Nginx는 실제 요청이 실패해야 그 서버를 빼고, 응답하지 않는 서버를 실패로 보기까지 기본 60초를 기다린다고 했습니다. 시뮬레이터의 ③ 탭에서 본 흰 점이 실제로 어떤 숫자로 나오는지 잽니다. 첫 번째 터미널에서 스크립트를 60초 동안 돌려 두고, 두 번째 터미널에서 10초쯤 뒤에 green을 멈춥니다.

직접 실행할 명령: green 멈추고 측정하기

```
# 첫 번째 터미널
python scripts/watch_requests.py http://localhost:8080 --seconds 60

# 두 번째 터미널 (스크립트를 켜고 10초쯤 뒤)
docker stop green
```

결과 해석: 기다리다 실패한 요청

green을 멈춘 직후부터 수십 초 동안 `실패: ReadTimeout` 줄이 여러 번 찍히고, 요약의 실패 수가 0이 아닙니다. 그 뒤로는 blue만 응답합니다. Nginx는 green에 연결을 시도한 요청이 60초(`proxy_connect_timeout`)를 다 기다려야 실패로 기록하는데, 그 기록이 남기 전까지는 green을 빼지 않으므로 그사이 green 차례가 된 요청들도 같이 기다립니다. 스크립트는 2

초 안에 답이 없으면 실패로 세므로 이 요청들이 모두 실패로 찍힙니다. 실제 사용자라면 화면이 멈춘 채로 기다리다 오류를 본 것입니다.

이번에는 연결을 기다리는 시간을 1초로 줄이고 같은 실험을 다시 합니다. Nginx는 설정을 읽을 때 컨테이너 이름을 IP 주소로 바꿔 두므로, green을 다시 켠 뒤에는 reload로 이름을 다시 찾게 합니다.

직접 실행할 명령: 연결 대기 시간을 1초로 줄이고 다시 측정하기 (macOS, Linux)

```
docker start green
cat > ~/bluegreen-lab/default.conf <<'EOF'
upstream active_app {
    zone active_app 64k;
    server blue:8080 weight=3;
    server green:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
        proxy_connect_timeout 1s;
    }
}
EOF
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload

# 첫 번째 터미널에서 60초 측정, 두 번째 터미널에서 10초쯤 뒤 docker stop green
python scripts/watch_requests.py http://localhost:8080 --seconds 60
```

Windows에서는 편집기로 `default.conf` 를 열어 위의 `cat` 과 `EOF` 사이 내용으로 바꿔 저장한 뒤, 나머지 줄을 실행합니다.

결과 해석: 다른 서버로 빨리 넘긴 요청

이번에는 green을 멈춰도 실패가 0건이거나 아주 적습니다. Nginx가 green에 1초 안에 연결되지 않으면 바로 실패로 보고 같은 요청을 blue로 다시 보내므로, 스크립트가 기다리는 2초 안에 blue의 응답이 도착하기 때문입니다. 대신 green 차례였던 요청은 1초쯤 늦게 응답합니다. 그리고 green을 뺀 뒤에도 `fail_timeout` (기본 10초)이 지나면 Nginx는 실제 요청 하나를 다시 green에 보내 보므로, green이 멈춰 있는 동안 10초마다 요청 하나씩은 1초를 더 기다립니다. 수동 헬스체크에서는 고장을 알아채는 비용을 사용자 요청이 낸다는 것이 이 숫자로 드러납니다.

대기 시간을 너무 짧게 잡으면 잠깐 바빠서 늦게 연결된 정상 서버까지 실패로 볼 수 있습니다. 실제 서비스에서는 서버가 보통 연결되는 시간보다 넉넉하게 잡거나, 능동 헬스체크를

하는 로드밸런서를 씁니다.

한 줄로 되돌리기

실험을 마쳤으면 green을 다시 켜고, 설정을 처음처럼 blue 한 줄로 되돌린 뒤 reload합니다.

직접 실행할 명령: 처음 설정으로 되돌리기 (macOS, Linux)

```
docker start green
cat > ~/bluegreen-lab/default.conf <<'EOF'
upstream active_app {
    zone active_app 64k;
    server blue:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
    }
}
EOF
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

Windows에서는 편집기로 `default.conf` 를 열어 위의 `cat` 과 `EOF` 사이 내용으로 바꿔 저장한 뒤, 나머지 줄을 실행합니다.

전환하면 사라지는 대화

이번에는 응답 코드가 아니라 대화를 봅니다. 지금 활성인 blue에서 화면을 열어 "내일 아침 8시에 물 마시라고 알려 줘"라고 보내고, 기억과 약속 화면에 약속이 생긴 것을 확인합니다. 그다음 green으로 전환하고 화면을 새로고침합니다.

직접 확인할 곳: 대화가 어디에 남는지 보기

1. blue가 활성인 상태에서 약속을 하나 만들고, 기억과 약속 화면에 보이는지 확인합니다.
2. green으로 전환하고 새로고침합니다. 방금 만든 약속과 대화가 보이는지 봅니다.
3. 다시 blue로 롤백하고 새로고침합니다. 약속이 다시 보이는지 봅니다.
4. `docker restart blue` 로 blue 컨테이너를 다시 시작하고 `docker exec bg-nginx nginx -s reload` 를 실행한 뒤 새로고침합니다. 약속이 남아 있는지 봅니다.

결과 해석: 상태가 컨테이너 안에 있을 때

green으로 전환하면 약속과 대화가 보이지 않고, blue로 돌아오면 다시 보입니다. 데이터가 사라진 것이 아니라 blue 컨테이너의 메모리에만 있었기 때문입니다. blue 컨테이너를 다시 시작하면 그 메모리도 비워져 약속이 완전히 사라집니다. 섹션 2에서 본 무상태 조건을 지금 에이전트 서비스가 채우지 못한다는 것을 직접 확인한 셈입니다. 새 버전을 배포할 때마다 대화가 사라지던 것도 같은 이유이고, 앞에서 두 줄로 나눠 보냈을 때처럼 컨테이너가 둘이면 요청이 어느 쪽으로 가느냐에 따라 보이는 대화가 달라집니다. 요청은 끊기지 않고 옮겨 갔지만, 사용자 입장에서는 친구가 방금 나눈 대화를 잊어버린 것이라 이 상태로는 무중단 배포라고 하기 어렵습니다.

주관식 퀴즈

Q. [이해 점검] 블루그린 전환 뒤 실패는 0건이었지만 대화가 사라졌고, green을 멈춘 실험에서는 대화와 상관없이 요청이 실패했습니다. 두 결과를 나란히 놓고, 무중단 배포를 확인할 때 "요청이 끊기지 않는 것"과 "사용자가 하던 일이 이어지는 것"을 왜 따로 봐야 하는지 설명해 보세요.

Chapter06

컨테이너 밖에 데이터 두기

섹션 5에서 blue와 green이 각자 메모리에 대화를 들고 있어서 전환하면 대화가 사라지는 것을 보았습니다. 데이터를 두 컨테이너가 함께 보는 데이터베이스로 옮기면, 컨테이너는 언제 꺼지고 새로 켜져도 되는 무상태가 됩니다.

이 과정에서는 데이터베이스로 Supabase를 씁니다. Supabase는 PostgreSQL 데이터베이스와 로그인 기능(Auth)을 함께 제공하는 관리형 서비스(managed service)로, 데이터베이스 서버의 설치, 업데이트, 복제와 백업 같은 운영을 Supabase가 맡습니다. 데이터를 사용자마다 나누려면 누가 보낸 요청인지 알아야 하므로 로그인도 함께 붙입니다. 처음에는 가입 없이 바로 쓰는 익명 로그인(anonymous sign-in)으로 시작해, 브라우저마다 게스트 사용자가 하나씩 생기게 합니다. 템플릿은 로그인 없이 데이터베이스만 켜면 모든 방문자의 대화가 한 사람 것으로 섞이므로, 그런 설정으로는 앱이 시작하지 않게 막아 두었습니다.

Supabase 프로젝트는 실제 사용자가 쓰는 서비스와 데이터가 섞이지 않게 시험용으로 하나 만듭니다. 무료 플랜은 활성 프로젝트가 2개까지이고, 1주 동안 활동이 없으면 일시정지됩니다(2026년 10월 기준). Supabase 계정은 에이전트 서비스 저장소가 있는 GitHub 계정으로 가입(Continue with GitHub)하면 계정을 따로 관리하지 않아도 됩니다. 절차는 템플릿의 `docs/supabase.md` 에도 같은 순서로 적혀 있습니다.

직접 확인할 곳: Supabase 시험용 프로젝트 만들기

1. Supabase 대시보드에서 프로젝트를 만듭니다. 이름은 `friend-agent-staging` 처럼 시험용임을 알 수 있게 짓고, 리전은 에이전트 서비스를 배포하는 Cloud Run 리전(us-central1)과 가까운 미국 리전(예: East US (Ohio))을 고릅니다. 데이터베이스 비밀번호는 Supabase가 만들어 주는 값을 비밀번호 관리 도구에 저장해 둡니다.
2. SQL Editor 에 템플릿 `supabase/migrations/0001_init.sql` 의 내용을 붙여 넣고 실행합니다. 캐릭터, 대화, 요약, 기억, 약속 테이블이 만들어지고, 모든 테이블에 RLS(Row Level Security, 행 단위 접근 제한)가 켜져 브라우저는 테이블을 직접 읽거나 쓸 수 없습니다.
3. Authentication > Sign In / Providers 에서 Allow anonymous sign-ins 를 켭니다.
4. Authentication > Rate Limits 에서 익명 로그인 한도를 확인합니다. 기본값은 IP 하나당 시간당 30건이라, 같은 와이파이를 쓰는 강의실에서는 금방 걸릴 수 있습니다.
5. Project Settings > API Keys 에서 프로젝트 URL, publishable key, secret key 가 어디 있는지 확인합니다. 키 값은 채팅이나 명령, 코드에 붙여 넣지 않습니다.

세 가지 값과 넣는 곳

Supabase에서 받은 값은 세 가지이고 성격이 다릅니다. 프로젝트 URL과 publishable key 는 브라우저가 로그인할 때 쓰는 값이라 화면 코드로 전달되고, 드러나도 RLS 때문에 테이블을 직접 읽을 수 없습니다. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터를 읽고 쓸 수 있는 키라서 서버에서만 씁니다. 내 컴퓨터에서는 세 값을 모두 저장소 루트의 `.env` 에 넣습니다. `.env` 는 커밋하지 않는 파일이라 secret key 를 넣어도 되지만, 화면에 출력하거나 채팅에 붙여 넣지 않습니다. 배포한 서비스에서는 secret key 를 LLM API 키처럼 시크릿 저장소(Secret Manager)에 두고 이름으로 연결합니다.

`.env` 에는 다섯 줄을 채웁니다. `STORAGE=supabase` 와 `AUTH=supabase` 는 저장소와 로그인을 Supabase로 바꾸는 스위치이고, 나머지는 `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_SECRET_KEY` 입니다. 값은 편집기로 `.env` 를 열어 직접 붙여 넣습니다.

코딩 에이전트에게 맡길 때: .env 에 넣을 줄 확인

```
.env 에서 Supabase 연결에 필요한 줄의 이름만 알려 줘. STORAGE, AUTH, SUPABASE_URL, SUPABASE_PU
BLISHABLE_KEY, SUPABASE_SECRET_KEY 가 맞는지 .env.example 과 비교해 줘.
값은 내가 편집기로 직접 넣을 거야. .env 파일의 내용을 읽어서 출력하지 마.
```

객관식 퀴즈

Q. Supabase에서 받은 값 가운데 secret key 만 서버에서 쓰고, publishable key 는 브라우저로 전달해도 되는 이유로 가장 알맞은 것은 무엇인가요?

• 1. secret key 가 더 길어서 브라우저 코드에 들어가지 않기 때문이다
• 2. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터에 접근할 수 있고, publishable key 는 드러나도 RLS 때문에 테이블을 직접 읽을 수 없기 때문이다
• 3. publishable key 는 Supabase가 매일 바꾸기 때문이다
• 4. 브라우저는 키를 하나만 쓸 수 있기 때문이다**제출**

- •

  1. secret key 가 더 길어서 브라우저 코드에 들어가지 않기 때문이다

- •

  2. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터에 접근할 수 있고, publishable key 는 드러나도 RLS 때문에 테이블을 직접 읽을 수 없기 때문이다

- •

  3. publishable key 는 Supabase가 매일 바꾸기 때문이다

- •

  4. 브라우저는 키를 하나만 쓸 수 있기 때문이다

데이터베이스를 붙여 다시 띄우기

이제 blue와 green을 `.env` 로 다시 띄웁니다. `--env-file .env` 는 `.env` 의 줄을 환경변수로 넣고, 뒤에 `-e` 로 준 값은 `.env` 보다 우선하므로 LLM은 여전히 페이크로, `K_REVISION` 과 `BANNER` 는 컨테이너마다 다르게 들어갑니다. 두 컨테이너는 같은 Supabase 프로젝트를 보게 되므로, 전환해도 대화가 이어져야 합니다. 컨테이너를 지우고 다시 띄우면 IP 주소가 바뀔 수 있으므로 마지막에 Nginx를 reload합니다.

직접 실행할 명령: 데이터베이스를 붙여 blue와 green 다시 띄우기 (저장소 루트에서)

```
# 메모리에 대화를 두던 두 컨테이너를 지운다
docker rm -f blue green

# .env 의 Supabase 값으로 다시 띄운다. -e 로 준 값이 .env 보다 우선한다
docker run -d --name blue  --network bg-lab --env-file .env -e LLM_PROVIDER=fake -e K_
REVISION=blue  -e BANNER=BLUE  friend-agent:local
docker run -d --name green --network bg-lab --env-file .env -e LLM_PROVIDER=fake -e K_
REVISION=green -e BANNER=GREEN friend-agent:local

# 바뀐 IP를 Nginx가 다시 찾게 한다
docker exec bg-nginx nginx -s reload
```

```
# 시작할 때 남긴 줄. storage 와 auth 가 supabase 인지 본다
docker logs blue 2>&1 | head -5
```

코딩 에이전트에게 맡길 때: 데이터베이스를 붙여 다시 띄우기

```
컨테이너 blue 와 green 을 지우고, 저장소 루트의 .env 를 --env-file 로 넣어 같은 이름으로 다시 띄워
줘.
LLM_PROVIDER=fake 는 그대로 두고, blue 는 K_REVISION=blue, BANNER=BLUE, green 은 K_REVISIO
N=green, BANNER=GREEN 을 -e 로 넣어 줘. 네트워크는 bg-lab, 포트는 열지 마.
다 띄운 뒤 bg-nginx 를 reload 하고, blue 로그의 처음 다섯 줄을 보여 줘. .env 의 값은 출력하지 마.
```

실행 전에 지우는 컨테이너가 `blue`, `green` 두 개뿐인지, `.env` 의 내용을 출력하는 명령(`cat .env` 등)이 없는지 봅니다. 로그 첫 줄의 `"storage": "supabase"`, `"auth": "supabase"` 와 `Uvicorn running` 이 보이면 정상입니다. "앱을 시작하지 않습니다"로 끝나는 줄이 보이면 그 줄에 적힌 값이 `.env` 에 빠진 것이므로 채운 뒤 다시 띄웁니다.

직접 확인할 곳: 전환해도 대화가 이어지는지 확인

1. 브라우저로 `http://localhost:8080` 을 열어 왼쪽 아래에 "게스트로 사용 중"이 보이는지 봅니다. 지금 upstream이 green이라면 먼저 blue로 바꿔 둡니다. "로그인을 시작하지 못했습니다" 창이 뜨면 대부분 Supabase의 Allow anonymous sign-ins 가 꺼져 있거나 저장되지 않은 것이므로, 켜고 저장한 뒤 새로고침합니다.
2. blue에서 캐릭터를 저장하고 약속을 하나 만든 뒤, Supabase 대시보드의 Table Editor 에서 `messages` 와 `reminders` 테이블에 행이 생겼는지 봅니다.
3. 섹션 5와 같은 방법으로 green으로 전환하고 새로고침합니다. 같은 약속과 대화가 그대로 보이는지 봅니다.
4. `docker restart blue` 와 `docker exec bg-nginx nginx -s reload` 를 실행한 뒤 blue로 롤백해도 약속이 남아 있는지 봅니다.
5. 다른 브라우저나 시크릿 창으로 `http://localhost:8080` 을 열면 다른 게스트 사용자로 시작해, 앞의 대화가 보이지 않는지 봅니다.

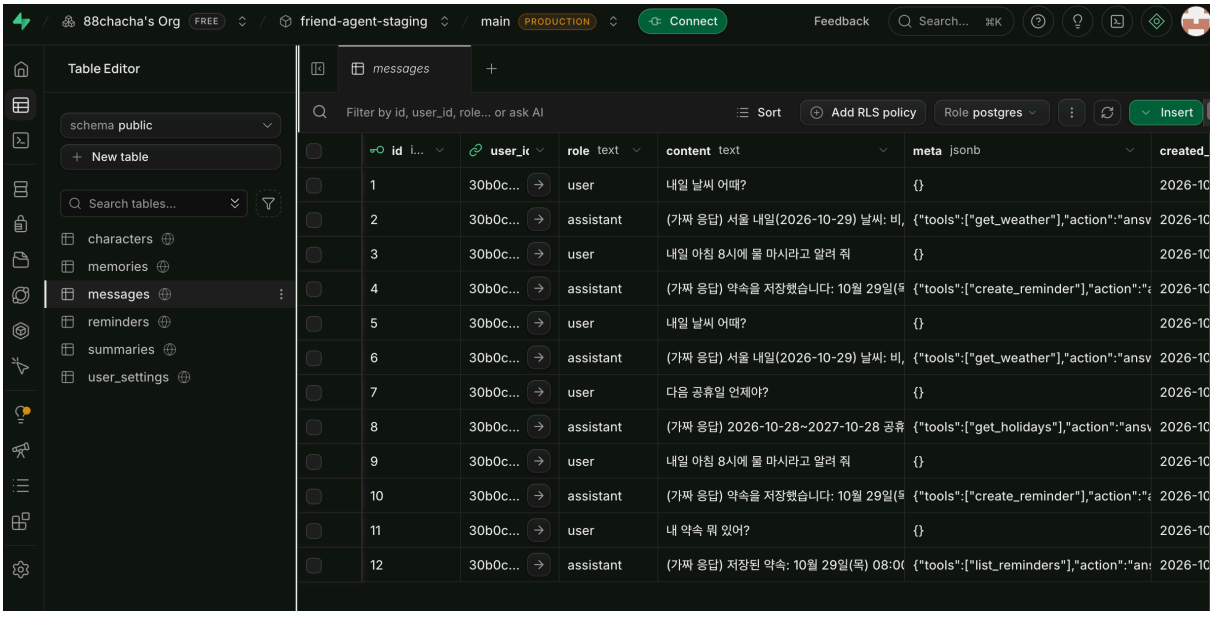


그림: Supabase 대시보드 > Table Editor > messages: 로컬 화면에서 보낸 메시지가 user_id, role, content 와 함께 행으로 쌓인 모습

결과 해석: 상태를 밖에 둔 서비스

이번에는 어느 쪽으로 전환해도, 컨테이너를 다시 시작해도 대화와 약속이 그대로입니다. 두 컨테이너가 같은 Supabase 테이블을 읽고 쓰고, 브라우저가 같은 게스트 사용자로 로그인해 있기 때문입니다. 다른 브라우저에서 앞의 대화가 보이지 않은 것은 브라우저마다 다른 게스트 사용자로 로그인하고, 서버가 모든 조회에 그 사용자의 ID 조건을 붙이기 때문입니다. 이제 blue와 green은 언제든 꺼지고 새로 켜져도 되는 무상태 서버가 되었고, 로드밸런서는 사용자가 알아채지 못하게 둘 사이를 오갈 수 있습니다. 섹션 5에서처럼 두 줄로 나눠 보내도 이제는 대화가 뒤섞이지 않습니다.

대신 두 버전이 같은 테이블을 함께 쓰므로, 한쪽 버전이 테이블 구조를 바꾸면 다른 쪽이 바로 영향을 받습니다. 섹션 2에서 본 DB 호환성과 확장-축소 패턴이 이제부터 실제로 적용됩니다.

객관식 퀴즈

Q. 데이터베이스를 붙인 뒤 blue에서 green으로 전환해도 대화가 이어졌습니다. 이때 새로 생기는 위험으로 가장 알맞은 것은 무엇인가요?

- 1. green이 blue의 메모리를 읽지 못한다
- 2. 두 버전이 같은 테이블을 함께 쓰므로, 한쪽이 테이블 구조를 바꾸면 전환 중이나 롤백 뒤에 다른 쪽이 깨질 수 있다
- 3. 로드밸런서가 데이터베이스 요청까지 나눠 보내서 느려진다
- 4. Supabase가 컨테이너마다 다른 데이터를 돌려준다**제출**

- •

  1. green이 blue의 메모리를 읽지 못한다

- •

  2. 두 버전이 같은 테이블을 함께 쓰므로, 한쪽이 테이블 구조를 바꾸면 전환 중이나 롤백 뒤에 다른 쪽이 깨질 수 있다

- •

  3. 로드밸런서가 데이터베이스 요청까지 나눠 보내서 느려진다

- •

  4. Supabase가 컨테이너마다 다른 데이터를 돌려준다

로컬 실습 정리

실습이 끝나면 로컬 컨테이너와 네트워크를 지웁니다. 지우기 전에 이름이 `blue`, `green`, `bg-nginx`, `bg-lab` 인지 확인합니다. Supabase 프로젝트와 `.env` 의 값은 지우지 않아도 됩니다. 이후 로컬에서 에이전트 서비스를 띄우면 같은 시험용 데이터베이스를 씁니다. 메모리 저장으로 돌아가려면 `.env` 의 `STORAGE` 와 `AUTH` 를 `memory`, `none` 으로 되돌립니다.

직접 실행할 명령: 로컬 실습 정리

```
docker rm -f blue green bg-nginx
docker network rm bg-lab
```

Chapter07

오늘 배운 것

고가용성의 기본. 가용성은 9의 개수로 말하고, 단일 장애점을 없애는 이중화, 고장을 알아채는 장애 감지, 예비로 넘기는 장애 조치로 고가용성을 만듭니다. 서버를 마음대로 늘리고 바꾸려면 상태를 서버 밖에 두는 무상태가 먼저 필요하고, 상태를 맡은 데이터베이스는 복제와 승격으로 이중화합니다.

무중단 배포와 로드밸런서. 롤링, 블루그린, 카나리는 모두 로드밸런서가 요청을 보내는 곳을 바꾸는 방법이고, 바꿀 때는 헬스체크, 그레이스풀 셧다운, DB 호환성을 지켜야 합니다. 로드밸런서는 능동 헬스체크나 수동 헬스체크로 고장 난 서버를 알아채는데, 무료 Nginx는 실제 요청이 실패해야 서버를 뺍니다.

직접 해 본 블루그린과 장애 실험. 내 컴퓨터에 Nginx와 에이전트 서비스 두 벌을 띄워, 설정 한 줄로 전환하고 롤백했습니다. 가중치로 요청을 3 대 1로 나눠 보냈고, 서버 하나를 멈췄을 때 연결 대기 시간에 따라 실패 수가 달라지는 것을 측정했습니다.

데이터를 밖으로. 전환하면 대화가 사라지는 것을 본 뒤 Supabase에 데이터와 익명 로그인을 붙이자, 어느 쪽으로 전환해도 대화가 이어졌습니다. 대신 두 버전이 같은 데이터베이스를 함께 쓰게 되었습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 새 버전으로 바뀌는 순간 사용자에게 일어날 수 있는 일을 적었습니다. 그 답을 다시 읽고, 오늘 직접 본 것(전환 중 실패 0건, 서버가 멈췄을 때의 실패, 사라진 대화, 데이터베이스를 붙인 뒤 이어진 대화)과 비교해 맞았던 것과 새로 알게 된 것을 적어 보세요.